

WorldSeed: Evolutionary Software Architecture Through PAC/SEC Dynamics

Applying Dawn Field Theory Principles to ML Architecture Evolution

Peter Lorne Groom
Dawn Field Institute
peter@dawnfield.ca

January 2026
Version 1.0 – DRAFT

Abstract

We explore whether principles from Dawn Field Theory—specifically Potential-Actualization Conservation (PAC) and Symbolic Entropy Collapse (SEC)—can guide the evolution of software and machine learning architectures. Through a series of experiments applying evolutionary dynamics to GAIA model architecture optimization, our preliminary results suggest promising directions for automated architecture design.

Core Investigation: Can physics-inspired information dynamics principles guide software evolution toward improved performance without manual architectural search?

Key Findings: Our experiments demonstrate (1) 3.8% fitness improvement over 5 generations of evolution, (2) 131% increase in token processing speed (335→776 tokens/second), (3) 27% improvement in output quality score (0.77→0.98), and (4) emergent discovery that increasing concentration threshold from ϕ^{-1} (0.618) to 0.785 improves performance.

Disclaimer: These findings represent preliminary computational explorations requiring independent validation, comparison with established neural architecture search methods, and extension beyond initial benchmarks. We present this work as an invitation for community investigation rather than established methodology.

Keywords: Dawn Field Theory, PAC, SEC, evolutionary architecture, WorldSeed, GAIA, software evolution, neural architecture search, Fibonacci structure

1 Introduction

1.1 The Architecture Search Problem

Modern machine learning requires choosing among vast configuration spaces: model size, training parameters, and architectural choices. Traditional approaches either rely on expert intuition or exhaustive search. Both have limitations: intuition doesn’t scale, and search is computationally expensive.

1.2 Biological Inspiration

Evolution has solved the architecture problem for billions of years. Organisms don’t search configuration space randomly—they inherit structure from parents, mutate variations, and face selection pressure. The fittest survive and reproduce.

Key insight from Dawn Field Theory experiments: Our prior work on digital life (exp_31) demonstrated that organisms with more Fibonacci-structured contacts harvest energy more efficiently and have survival advantages. This suggested coherence-based fitness might apply beyond biology.

1.3 Research Questions

This exploration investigates:

1. **Can PAC/SEC dynamics guide software evolution?** Do conservation principles and coherence metrics provide meaningful selection pressure?
2. **Does template-guided generation improve convergence?** Like biological inheritance, does starting from parent structure accelerate evolution?
3. **Do theoretical constants emerge during evolution?** Do ϕ (golden ratio) and Ξ (balance constant) appear in optimized configurations?
4. **Is evolution competitive with manual design?** Can evolved architectures match or exceed hand-tuned baselines?

1.4 Contributions

This paper presents:

1. **WorldSeed Framework:** A complete system for evolutionary software/ML architecture design based on PAC/SEC principles
2. **GAIA Evolution Results:** Preliminary validation showing 3.8% fitness improvement with 131% speed increase on WikiText-2
3. **Mutation Taxonomy:** Eight concrete mutation types for ML architecture evolution
4. **Emergent Insights:** Discovery that evolution favors stricter quality gates (concentration threshold $0.618 \rightarrow 0.785$)
5. **Reproduction Package:** Complete code, data, and figures for independent validation

2 Theoretical Background

2.1 PAC: Potential-Actualization Conservation

PAC formalizes the principle that information value conserves across decomposition:

$$f(\text{parent}) = \sum_{u \in D(\text{parent})} \alpha_{v \rightarrow u} \cdot f(u) \quad (1)$$

In software evolution, this manifests as **architectural invariants**: total functionality conserves across module decomposition, interface contracts preserved across generations, and system identity maintained through mutations.

2.2 SEC: Symbolic Entropy Collapse

SEC describes how high-entropy exploration states crystallize into stable structures:

$$\frac{\partial S}{\partial t} = \alpha \nabla I - \beta \nabla H \quad (2)$$

Where S = structure formation rate, ∇I = information gradient (drives crystallization), and ∇H = entropy gradient (drives exploration).

In software evolution, SEC provides **selection pressure**: high-coherence components survive, low-coherence components die, with balance between exploration and exploitation.

2.3 MED: Macro Emergence Dynamics

MED predicts bounded complexity in emergent systems:

All complex flows converge to symbolic patterns with depth ≤ 2 and nodes ≤ 3 .

In software evolution, MED provides **architectural bounds**: maximum hierarchy depth (Fibonacci-constrained), maximum components per level, and natural stopping criteria for growth.

2.4 Mapping to Software Evolution

DFT Principle	Biological Analog	Software Analog
PAC Conservation	Genetic inheritance	Architectural invariants
SEC Coherence	Fitness landscape	Coherence metrics
SEC Threshold	Survival threshold	Quality gate
MED Bounds	Organism complexity	Architecture depth
ϕ (golden ratio)	Growth patterns	Fibonacci structure
Ξ (balance constant)	Metabolic efficiency	Performance balance

Table 1: Mapping Dawn Field Theory principles to software evolution

3 WorldSeed Architecture

3.1 System Overview

WorldSeed implements evolutionary dynamics for software/ML architecture design. The system takes a seed specification (defining PAC invariants, SEC dynamics, and mutable parameters) and evolves it through generations of mutation, evaluation, and selection.

3.2 Mutation System

Eight mutation types provide structured variation:

Mutation	Parameter	Range	Rationale
Context	context_size	3–13 (Fibonacci)	Attention window
Hot Contexts	hot_contexts	5K–50K	Memory capacity
Concentration	threshold	0.5–0.8	Quality gate
Embedding	embedding_dim	256–1024	Representation capacity
Top-K	top_k	50–500	Generation diversity
Reject	reject_attempts	1–10	Quality enforcement
Phi	ϕ	$\pm 5\%$ around 1.618	PAC balance
Xi	Ξ	$\pm 5\%$ around 1.057	SEC balance

Table 2: Mutation types for ML architecture evolution

3.3 Fitness Function

Multi-objective fitness balances four concerns:

$$\text{fitness} = 0.30 \cdot \text{perplexity} + 0.30 \cdot \text{speed} + 0.20 \cdot \text{memory} + 0.20 \cdot \text{quality} \quad (3)$$

Generation-dependent weighting shifts emphasis over time:

- **Gen 0–2:** 70% coherence, 30% tests (filter garbage)
- **Gen 3–9:** 50% coherence, 50% tests (balance)
- **Gen 10+:** 30% coherence, 70% tests (optimize benchmarks)

3.4 Selection and Genealogy

Selection: Greedy per generation—multiple candidates compete (3 per generation), 66% mortality rate creates selection pressure, only fittest advances.

Genealogy Tracking: Full provenance tree with parent ID, mutation history, and fitness at each generation.

4 Experimental Setup

4.1 GAIA Model Background

GAIA (Generalized Architectures for Intelligent Actualization) is a backprop-free language model using grafted embeddings from GPT-2, PAC tree with delta compression, sparse transition matrix, and concentration-based quality gates.

4.2 Dataset and Configuration

WikiText-2 benchmark with 20K token training subset for quick tests. Evolution configuration: 5 generations, 3 candidates per generation, 0.50 coherence threshold.

Baseline: Manual GAIA configuration with `context_size=5`, `concentration=0.618`, `embedding_dim=768`, $\phi=1.618$, $\Xi=1.057$.

5 Results

5.1 Evolution Trajectory

Generation	Best Fitness	Mean Fitness	Improvement	Key Mutation
Baseline	1.445	–	0.0%	–
1	1.466	1.454	+1.4%	reject_attempts
2	1.465	1.461	+1.4%	embedding_dim, ϕ
3	1.499	1.470	+3.8%	Ξ
4	1.502	1.480	+3.9%	top_k
5	1.500	1.470	+3.8%	embedding_dim, top_k

Table 3: Evolution trajectory over 5 generations

Key observation: Generation 3 showed a 2.4% jump when Ξ was mutated, suggesting this parameter significantly impacts GAIA performance.

5.2 Performance Comparison

Metric	Baseline	Evolved	Change
Overall Fitness	1.445	1.500	+3.8%
Speed (tok/s)	335	776	+131%
Quality Score	0.77	0.98	+27%
Memory (MB)	156	156	0%

Table 4: Performance comparison: baseline vs. evolved configuration

5.3 Emergent Insights

Discovery 1: Higher Concentration Threshold—Evolution increased threshold from 0.618 (ϕ^{-1}) to 0.785, a 27% increase. The system discovered that stricter quality gates improve overall fitness.

Discovery 2: Embedding Dimension Reduction—Evolution reduced embedding_dim from 768 to 512, improving speed while maintaining quality.

Discovery 3: Top-K Increase—Evolution doubled top_k from 100 to 200, creating a “generate more, filter harder” strategy.

5.4 Constant Tracking

Constant	Baseline	Final	Theory	Error
ϕ (phi)	1.618	1.560	1.618	3.5%
Ξ (xi)	1.057	1.010	1.057	4.5%

Table 5: Constant evolution across generations

6 Analysis

6.1 What the Results Might Suggest

These preliminary results suggest several possibilities:

1. **PAC/SEC dynamics may provide meaningful selection pressure** for software evolution. Coherence-based fitness appears to guide evolution toward improved performance.
2. **Template-guided generation appears to accelerate convergence.** By inheriting from parent configurations, evolution avoids random search from scratch.
3. **Multi-objective fitness may discover non-obvious trade-offs.** The evolved configuration trades embedding dimension for speed while compensating with increased top-k diversity.

6.2 Alternative Explanations

Several alternative explanations merit consideration:

1. **Random search baseline:** Improvements might be achievable through random configuration sampling.

2. **Overfitting to fitness function:** Evolved configuration optimizes our specific fitness function, which may not generalize.
3. **Limited exploration:** With only 5 generations and 3 candidates each, the search space explored is small.

6.3 Limitations

Computational: Quick test only (5 generations, 20K tokens), single run (no statistical replication).

Methodological: No comparison with NAS/AutoML baselines, no downstream task evaluation, single model family.

Theoretical: Constant divergence suggests insufficient data, fitness weights chosen heuristically.

7 Related Work

7.1 Neural Architecture Search

DARTS [1]: Differentiable architecture search using continuous relaxation.

NASNet [2]: Reinforcement learning for architecture search.

WorldSeed differs in using physics-inspired coherence metrics rather than pure benchmark optimization.

7.2 Genetic Programming

Cartesian GP [3]: Evolving program graphs.

WorldSeed applies GP principles to ML architecture with domain-specific fitness functions.

8 Conclusion

We explored whether PAC/SEC dynamics from Dawn Field Theory could guide evolutionary software architecture design. Preliminary experiments suggest:

- 3.8% fitness improvement over 5 generations
- 131% speed improvement through evolved embedding reduction
- 27% quality improvement via discovered concentration threshold
- Emergent “generate more, filter harder” optimization strategy

If validated, this approach suggests physics principles may extend to software design, combining evolutionary mechanics with information-theoretic fitness for interpretable architecture search.

8.1 Future Directions

1. Scale experiments: Full WikiText-2, more generations, statistical replication
2. Baseline comparisons: Compare with NAS, random search, Bayesian optimization
3. Generalization tests: Multiple model families, multiple datasets
4. Neural network extension: Apply WorldSeed to transformer architecture search

8.2 Open Science Commitment

All code, data, and experimental protocols are available in our open-source repository. We encourage independent replication, critique, and extension of this work.

References

- [1] Liu, H., Simonyan, K., & Yang, Y. (2019). DARTS: Differentiable architecture search. *ICLR*.
- [2] Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. *CVPR*.
- [3] Miller, J. F. (2011). Cartesian genetic programming. *Springer*.
- [4] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI*.
- [5] Biderman, S., et al. (2023). Pythia: A suite for analyzing large language models across training and scaling. *ICML*.

A Full Evolution Results

Best evolved configuration:

```
{
  "embedding_dim": 512,
  "context_size": 5,
  "hot_contexts": 10000,
  "top_k_per_context": 200,
  "concentration_threshold": 0.785,
  "phi": 1.560,
  "xi": 1.010,
  "fitness": 1.500,
  "improvement": 3.8%
}
```

B Reproduction Instructions

```
# Install dependencies
pip install -r Code/requirements.txt

# Run evolution
python Code/experiments/exp_03_wikitext2_evolution.py

# Expected: Data/results/evolution_results.json
```